



Dictionaryes

Practical Skills Module 2: Topic 8

[Visit our website](#)

Practical Skills Module 2: Topic 8

Topic Code	PM-02-PS08
Topic	Use dictionaries in Python to store data values

Scope of Practical Skill

Given an applicable instruction and access to a learning platform, the learner must be able to:

- PA0801 Create a dictionary
- PA0802 Access elements
- PA0803 Add and remove elements
- PA0804 Check if a given key already exists in a dictionary
- PA0805 Add Key/Value Pair
- PA0806 Update dictionary
- PA0807 Merge dictionaries

Applied Knowledge

- AK0801 Concept, definition and functions of Python data types

Internal Assessment Criteria

- IAC0801 Expected results with Python dictionaries are achieved

Resources:

Knowledge	Information from Python Programmer Curriculum
National Curriculum Framework	900221-000-00-PM-02



Practical task

Follow the facilitator's instructions to complete the following activities:

Scenario: Student Management System

In this scenario, a school needs a Python program to manage student data. The program will use dictionaries to store information about students, such as their names, grades, and subjects. You will perform various operations like creating dictionaries, accessing and updating student information, adding or removing students, and merging records.

Create a Python file called **student_management.py** and complete the steps below:

Step 1: Create a dictionary of student data.

- Create a dictionary where:
 - Each **key** is a unique student ID (e.g., "S001")
 - Each **value** is another dictionary with:
 - "name": the student's name
 - "grade": the student's grade
 - "subjects": a list of subjects

Example structure:

```
{
    "S001": {"name": "Zara", "grade": "A", "subjects": ["Math", "English"]},
    "S002": {"name": "Liam", "grade": "B", "subjects": ["Science",
"History"]}
}
```

Step 2: Access a student's details by ID.

- Use the student ID to access and print all details for a specific student.
- Use a `print()` statement with labels to clearly show their information.
 - *Example output:*
Student Name: Zara, Grade: A, Subjects: ['Math', 'English']

Step 3: Add a new student and remove one.

- Add a new student record to the dictionary using a new student ID.
- Then remove an existing student using their student ID with the `del` keyword.
- Print the dictionary to confirm the changes.

Step 4: Check for a student ID before adding/modifying.

- Use an `if` statement to check whether a specific student ID **already exists** before:
 - Adding a new student
 - Modifying an existing one
- Print a message indicating whether the student ID was found.
 - *Example output:* Student ID S003 already exists. or Student ID S005 not found.

Step 5: Add a new key-value pair to a student record.

- Add a new key called "**attendance**" to a selected student's dictionary.
- Set the value as a percentage (e.g., 95.5) to represent attendance.

Step 6: Update grade and add a subject.

- Change a student's "**grade**" value to a new grade.
- Add a new subject to the "**subjects**" list using an appropriate list method.

Tip: Append to the existing list of subjects without replacing it.

Step 7: Merge in new student records.

- Create a second dictionary containing **two more students** with the same structure as the original.
- Merge the new dictionary into the original one using the `update()` method.
- Print the final dictionary to show all student records combined.



Take note

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.
